Programa:	Icontoy 3.1
Protección:	Trial 30 días
Objetivos:	Obtener un serial válido
URL descarga:	http://www.lighttek.com/
Dificultad:	Más sencillo no lo encontraremos ;-)
Herramientas:	OllyDbg v1.10 + PeiD 0.93
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 12-11-05

Hola a todos los CracksLatinoS una vez más..!!

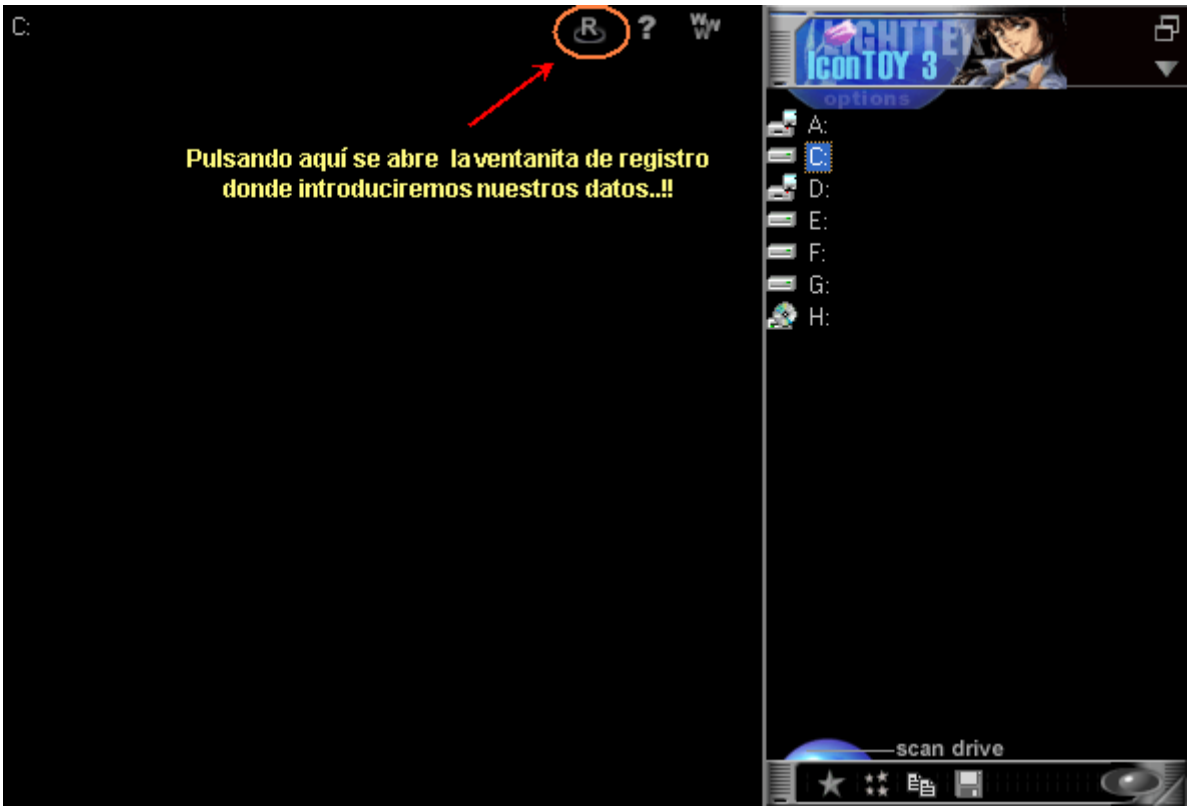
No me cansaré de escribir tutes, jeje… es una de las cosas que más me relajan y reconfortan el crackear una protección y escreibir sobre ella, me da igual si es fácil, mediana, complicada, etc., lo siempre me reconforta, en este caso vamos a ver una de las protecciones más sencillas que existen: los hardcoded serial ¿y qué demonios es eso? Un hardcoded serial significa que el serial válido se encuentra en las propias instrucciones del programa, es decir, en el código, podemos verlo con W32dasm o bien bajo Olly ya que a veces no está muy claro como es el caso que tratamos. Antes de continuar miraremos si el objetivo (target, jeje…) está empakadi y/o encriptado:

Borland Delphi 4.0 - 5.0

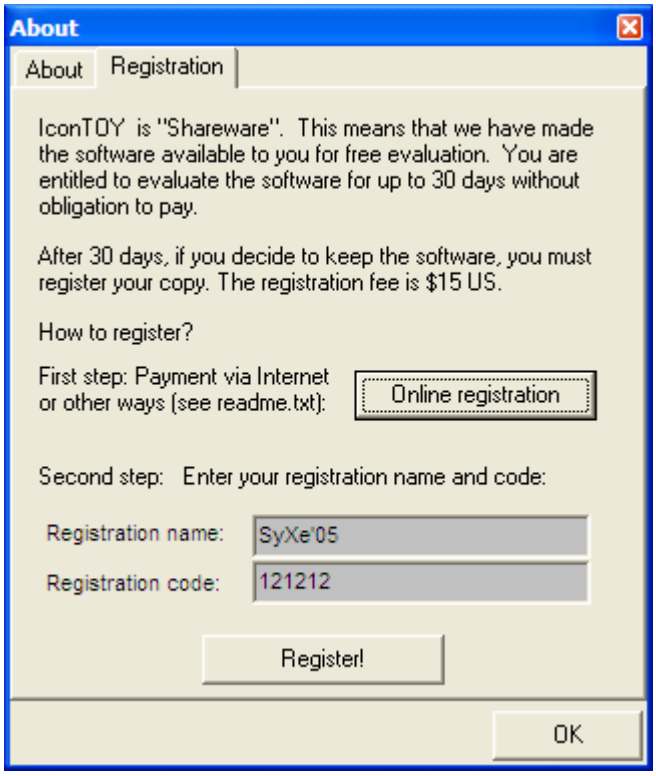
Pues no lo está como se ve en la imagen anterior, está compilado en Delphi, entonces podemos atacarlo con Olly sin problema alguno ;-)

Abrimos entonces un OllyDbg y cargamos el programa, si pulsamos F9 vemos que corre bien, descartamos pues algún truco antidebug, pero vamos a lo que vamos…

Arrancamos el programa y nos aparece esta ventana en el escritorio:



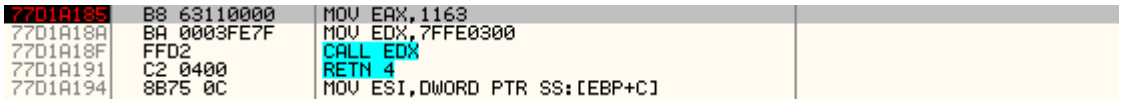
Es el programa principal por su puesto, si picamos en el botón de la "R" que remarco en la imagen se abrirá una ventanita de registro de esas típicas donde debemos introducir un nombre y un serial como casi siempre:



Cuando tenemos nuestros datos puestos como se ve en la imagen pulsamos el botón "Register" para que valide el serial, seguramente se generará uno distinto para cada nombre de usuario verdad? Eso pensamos casi siempre que vemos una ventanita de este tipo pero nos vamos a llevar más de una sorpresa ;-) de momento aceptamos y…

Mensaje de error al canto, jeje.. nos hemos confundido de serial parece ser, bueno no pasa absolutamente nada, seguiremos a lo nuestro, esta ventanita parece además al típico MessageBoxA, podemos poner un BP en dicha API pero no para ¿? Es evidente que no la está usando, quizá utilice otra similar que no es MessageBoxW porque lo he probado, entonces se me ocurre utilizar una técnica que muy pocas veces falla: DestroyWindow, jeje.. el sistema debe ver el mensaje como una ventana (window) esto quiere decir que cuando pulsemos el botón "OK" esta se cerrará verdad? Y como lo hará estamos pensando, pues casi con total seguridad por medio de la API DestroyWindow que se utiliza para eso precisamente como su nombre indica, entonces lo que hacemos es lo siguiente: con el mensaje en pantalla ponemos el BP:

Si no se queja el BP queda puesto, luego ya podemos pulsar en OK para que se cierre la ventana, en este momento Olly se detiene aquí:



Anque estamos lejos todavía de lo que buscamos, estamos dentro de la librería User32.dll y debemos salir primero al código del programa mediante Ctrl+F9 cada vez, al final llegamos pero resulta que tampoco es la zona adecuada que estamos buscando, esto se sabe por varios motivos, uno de ellos es que antes del CALL no se vean saltos condicionales que lo eviten, otra es que el código sea reconocido por nosotros y ya sepamos que no estamos aún, etc., ya digo que son varios pero nos llevan todos a lo mismo: a seguir traceando a lo retro que se llama pues vamos para atrás en busca de la

llamada padre, o sea que estamos en algún subcall de la rutina de verificación para entendernos, al final obtenemos recompensa, veamos:

```
00486021  .   8BC3          MOV EAX,EBX
00486023  .   E8 448AFDFF   CALL icontoy.0045EA6C
00486028  .   8BC3          MOV EAX,EBX
0048602A  .   E8 1587FDFF   CALL icontoy.0045E744
0048602F  .   8BC3          MOV EAX,EBX
00486031  .   E8 02CFF7FF   CALL icontoy.00402F38
00486036  .   B8 F4604800   MOV EAX,icontoy.004860F4          ASCII "You are registered! Thank you."
0048603B  .   E8 4080FCFF   CALL icontoy.0044E080
00486040  .   8D45 F4       LEA EAX,DWORD PTR SS:[EBP-C]
00486043  .   8B0D 08E9480  MOV ECX,DWORD PTR DS:[48E908]
00486049  .   BA 1C614800   MOV EDX,icontoy.0048611C          ASCII "Registered for: "
0048604E  .   E8 7DDDF7FF   CALL icontoy.00403DD0
00486053  .   8B55 F4       MOV EDX,DWORD PTR SS:[EBP-C]
00486056  .   A1 FCE84800   MOV EAX,DWORD PTR DS:[48E8FC]
0048605B  .   8B80 E802000  MOV EAX,DWORD PTR DS:[EAX+2E8]
00486061  .   E8 D654FAFF   CALL icontoy.0042B53C
00486066  .   A1 80D24800   MOV EAX,DWORD PTR DS:[48D280]
0048606B  .   8B00          MOV EAX,DWORD PTR DS:[EAX]
0048606D  .   B2 01         MOV DL,1
0048606F  .   8B08          MOV ECX,DWORD PTR DS:[EAX]
00486071  .   FF51 60       CALL DWORD PTR DS:[ECX+60]
00486074  .v  EB 0A         JMP SHORT icontoy.00486080
00486076  >   B8 38614800   MOV EAX,icontoy.00486138          ASCII "Registration key error!"
0048607B  .   E8 0080FCFF   CALL icontoy.0044E080
00486080  >   33C0          XOR EAX,EAX
00486082  .   5A            POP EDX
00486083  .   59            POP ECX
```

12 RET's conté hasta llegar aquí, por eso digo que a veces hay que tener paciencia, en otros casos al 4° ó 5° RET ya estamos en la zona pero aquí no es así, lo importante es que al final llegamos a la zona clave de todo este asunto, como vemos hay un JMP en :486074 que evita pasar por el CALL de :48607B sonde nos saca el mensaje de error, entonces ¿cómo hemos llegado? La respuesta está algo más arriba donde debe existir un salto condicional que apunte justo debajo de este JMP, si nos situamos en :486076 vemos:

```
00486049  .   BA 1C614800   MOV EDX,icontoy.0048611C          ASCII "Registered for: "
0048604E  .   E8 7DDDF7FF   CALL icontoy.00403DD0
00486053  .   8B55 F4       MOV EDX,DWORD PTR SS:[EBP-C]
00486056  .   A1 FCE84800   MOV EAX,DWORD PTR DS:[48E8FC]
0048605B  .   8B80 E802000  MOV EAX,DWORD PTR DS:[EAX+2E8]
00486061  .   E8 D654FAFF   CALL icontoy.0042B53C
00486066  .   A1 80D24800   MOV EAX,DWORD PTR DS:[48D280]
0048606B  .   8B00          MOV EAX,DWORD PTR DS:[EAX]
0048606D  .   B2 01         MOV DL,1
0048606F  .   8B08          MOV ECX,DWORD PTR DS:[EAX]
00486071  .   FF51 60       CALL DWORD PTR DS:[ECX+60]
00486074  .v  EB 0A         JMP SHORT icontoy.00486080
00486076  >   B8 38614800   MOV EAX,icontoy.00486138          ASCII "Registration key error!"
0048607B  .   E8 0080FCFF   CALL icontoy.0044E080
00486080  >   33C0          XOR EAX,EAX
00486082  .   5A            POP EDX
```

Esa flechita roja indica que efectivamente hay un salto apuntando a la zona, además también el símbolo ">" que hay al lado izquierdo de la instrucción indica lo mismo, nos desplazamos hacia arriba en busca del salto:

```
00485F74  .   E8 9355FAFF   CALL icontoy.0042B50C
00485F79  .   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]
00485F7C  .   E8 6725F8FF   CALL icontoy.004084E8             ;pone en EAX el serial en hexa...
00485F81  .   A3 04E94800   MOV DWORD PTR DS:[48E904],EAX
00485F86  .   33C0          XOR EAX,EAX
00485F88  .   5A            POP EDX
00485F89  .   59            POP ECX
00485F8A  .   59            POP ECX
00485F8B  .   64:8910       MOV DWORD PTR FS:[EAX],EDX
00485F8E  .   68 9B5F4800   PUSH icontoy.00485F9B
00485F93  >   C3            RETN                              RET used as a jump to 00485F9B
00485F94  .^  E9 13D6F7FF   JMP icontoy.004035AC
00485F99  .^  EB F8         JMP SHORT icontoy.00485F93
00485F9B  >   A1 04E94800   MOV EAX,DWORD PTR DS:[48E904]
00485FA0  .   E8 B3FCFFFF   CALL icontoy.00485C58             ;verifica el serial...
00485FA5  .   803D 00E9480  CMP BYTE PTR DS:[48E900],1
00485FAC  .v  0F85 C400000  JNZ icontoy.00486076               ;salto a invalid key...
00485FB2  .   8D55 F8       LEA EDX,DWORD PTR SS:[EBP-8]
00485FB5  .   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
```

Ahí lo tenemos, verifica la variable global [48E900] en caso de contener un valor distinto de 0 salta hacia el mensaje de error, está claro que debe valer 0, si miramos por encima encontramos el inicio de la rutina en :485F38 allí ponemos un BP y reintroducimos los datos, aceptamos y Olly para allí, traceamos un poco hasta llegar a un CALL interesante:

```
00485F63  .   A3 04E94800   MOV DWORD PTR DS:[48E904],EAX
00485F68  .   8D55 F8       LEA EDX,DWORD PTR SS:[EBP-8]
00485F6B  .   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
00485F6E  .   8B80 1003000  MOV EAX,DWORD PTR DS:[EAX+310]
00485F74  .   E8 9355FAFF   CALL icontoy.0042B50C
00485F79  .   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]
00485F7C  .   E8 6725F8FF   CALL icontoy.004084E8             ;pone en EAX el serial en hexa...
00485F81  .   A3 04E94800   MOV DWORD PTR DS:[48E904],EAX
00485F86  .   33C0          XOR EAX,EAX
```

Si en este instante miramos los registros nos dicen ya algo:

```
Registers (FPU)
EAX 00A0C498 ASCII "121212"
ECX 77D13B4B user32.77D13B4B
EDX 00140608
EBX 00A144DC
ESP 0012F680
EBP 0012F6A8
ESI 0012F820
EDI 0012F820
```

EAX está apuntando a nuestro serial MALO, jeje… entramos en el CALL en el que estamos a ver qué hace con él y llegamos a este loop:

```
00402CC2  |>  80EB 30        /SUB BL,30
00402CC5  |. |80FB 09        |CMP BL,9
00402CC8  |. |77 2C          |JA SHORT icontoy.00402CF6
00402CCA  |. |39F8           |CMP EAX,EDI
00402CCC  |. |77 28          |JA SHORT icontoy.00402CF6
00402CCE  |. |8D0480         |LEA EAX,DWORD PTR DS:[EAX+EAX*4]
00402CD1  |. |01C0           |ADD EAX,EAX
00402CD3  |. |01D8           |ADD EAX,EBX
00402CD5  |. |8A1E           |MOV BL,BYTE PTR DS:[ESI]
00402CD7  |. |46             |INC ESI
00402CD8  |. |84DB           |TEST BL,BL
00402CDA  |.^\75 E6          \JNZ SHORT icontoy.00402CC2
```

donde lo único que hace es pasar el serial en formato hexa a EAX, quedando EAX==1D97C aunque no lo parezca este es nuestro serial, hagamos la prueba:

```
Command : 1d97c                HEX: 1D97C - DEC: 121212 - ASCII: ·Ù|
```

Se ve claro como representa a nuestro serial "121212" así que el CALL lo único que hace es pasar el serial en formato hexa al registro EAX ¿? Sigamos a ver qué más encontramos:

```
00485F81   .  A3 04E94800   MOV DWORD PTR DS:[48E904],EAX
```

Esto me parece interesante, le pasa el serial a la veriable [48E904] que después recupera antes de este CALL (el verificador, jeje…)

```
00485F94  .^ E9 13D6F7FF  JMP icontoy.004035AC
00485F99  .^ EB F8        JMP SHORT icontoy.00485F93
00485F9B  > A1 04E94800   MOV EAX,DWORD PTR DS:[48E904]      ;recupera serial...
00485FA0  .  E8 B3FCFFFF  CALL icontoy.00485C58              ;verifica el serial...
00485FA5  .  803D 00E9480 CMP BYTE PTR DS:[48E900],1
00485FAC  .v 0F85 C400000 JNZ icontoy.00486076               ;salto a invalid key...
00485FB2  .  8D55 F8      LEA EDX,DWORD PTR SS:[EBP-8]
```

Pues ya está más que claro que el "CALL 485C58" es el call_verificador y de él depende que salte en :485FAC aunque no lo parezca, entramos en dicho CALL con F7 a ver qué hace con EAX que recordemos tiene nuestro serial, queeeeeeeeeeeeee?

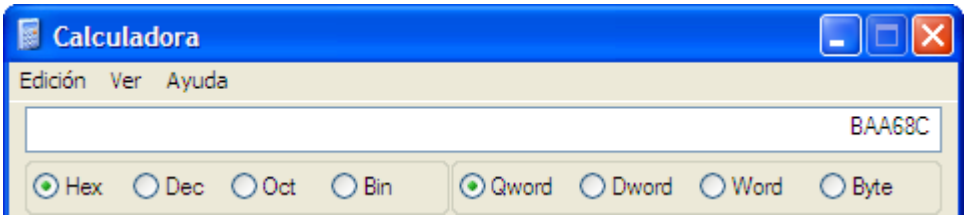
```
00485C58  $  55           PUSH EBP
00485C59  .  8BEC         MOV EBP,ESP
00485C5B  .  6A 00        PUSH 0
00485C5D  .  33D2         XOR EDX,EDX
00485C5F  .  55           PUSH EBP
00485C60  .  68 CD5E4800  PUSH icontoy.00485ECD
00485C65  .  64:FF32      PUSH DWORD PTR FS:[EDX]
00485C68  .  64:8922      MOV DWORD PTR FS:[EDX],ESP
00485C6B  .  3D 8CA6BA00  CMP EAX,0BAA68C
00485C70  .v 0F84 B901000 JE icontoy.00485E2F
00485C76  .  3D 6A5A1300  CMP EAX,135A6A
00485C7B  .v 0F84 AE01000 JE icontoy.00485E2F
00485C81  .  3D E0621300  CMP EAX,1362E0
00485C86  .v 0F84 A301000 JE icontoy.00485E2F
00485C8C  .  3D 76281300  CMP EAX,132876
00485C91  .v 0F84 9801000 JE icontoy.00485E2F
00485C97  .  3D AEED3100  CMP EAX,31EDAE
00485C9C  .v 0F84 8D01000 JE icontoy.00485E2F
00485CA2  .  3D C6E93100  CMP EAX,31E9C6
00485CA7  .v 0F84 8201000 JE icontoy.00485E2F
00485CAD  .  3D A8D81200  CMP EAX,12D8A8
00485CB2  .v 0F84 7701000 JE icontoy.00485E2F
00485CB8  .  3D 267F1300  CMP EAX,137F26
00485CBD  .v 0F84 6C01000 JE icontoy.00485E2F
00485CC3  .  3D B6B7B800  CMP EAX,0B8B7B6
00485CC8  .v 0F84 6101000 JE icontoy.00485E2F
00485CCE  .  3D AE7A1200  CMP EAX,127AAE
00485CD3  .v 0F84 5601000 JE icontoy.00485E2F
00485CD9  .  3D A2D21200  CMP EAX,12D2A2
00485CDE  .v 0F84 4B01000 JE icontoy.00485E2F
00485CE4  .  3D 2AFAB900  CMP EAX,0B9FA2A
00485CE9  .v 0F84 4001000 JE icontoy.00485E2F
00485CEF  .  3D 36F8B900  CMP EAX,0B9F836
00485CF4  .v 0F84 3501000 JE icontoy.00485E2F
00485CFA  .  3D 0810C100  CMP EAX,0C11008
00485CFF  .v 0F84 2A01000 JE icontoy.00485E2F
```

```
00485E13  .   3D 02AC2600   CMP EAX,26AC02
00485E18  .v  74 15         JE SHORT icontoy.00485E2F
00485E1A  .   3D 9C0B8101   CMP EAX,1810B9C
00485E1F  .v  74 0E         JE SHORT icontoy.00485E2F
00485E21  .   3D D0128B01   CMP EAX,18B12D0
00485E26  .v  74 07         JE SHORT icontoy.00485E2F
00485E28  .   3D 24888101   CMP EAX,1818824
00485E2D  .v  75 7C         JNZ SHORT icontoy.00485EAB
00485E2F  >   C605 00E9480  MOV BYTE PTR DS:[48E900],1
00485E36  .   A1 FCE84800   MOV EAX,DWORD PTR DS:[48E8FC]
00485E3B  .   8B80 D002000  MOV EAX,DWORD PTR DS:[EAX+2D0]
00485E41  .   33D2          XOR EDX,EDX
00485E43  .   E8 18C0FCFF   CALL icontoy.00451E60
00485E48  .   A1 FCE84800   MOV EAX,DWORD PTR DS:[48E8FC]
00485E4D  .   8B90 CC02000  MOV EDX,DWORD PTR DS:[EAX+2CC]
00485E53  .   A1 FCE84800   MOV EAX,DWORD PTR DS:[48E8FC]
00485E58  .   8B80 C802000  MOV EAX,DWORD PTR DS:[EAX+2C8]
00485E5E  .   E8 E5C6FCFF   CALL icontoy.00452548
00485E63  .   8D45 FC       LEA EAX,DWORD PTR SS:[EBP-4]
00485E66  .   8B0D 08E9480  MOV ECX,DWORD PTR DS:[48E908]
00485E6C  .   BA E05E4800   MOV EDX,icontoy.00485EE0        ASCII "Registered for: "
00485E71  .   E8 5ADFF7FF   CALL icontoy.00403DD0
00485E76  .   8B55 FC       MOV EDX,DWORD PTR SS:[EBP-4]
00485E79  .   A1 FCE84800   MOV EAX,DWORD PTR DS:[48E8FC]
00485E7E  .   8B80 E802000  MOV EAX,DWORD PTR DS:[EAX+2E8]
00485E84  .   E8 B356FAFF   CALL icontoy.0042B53C
00485E89  .   A1 80D24800   MOV EAX,DWORD PTR DS:[48D280]
00485E8E  .   8B00          MOV EAX,DWORD PTR DS:[EAX]
00485E90  .   8B80 D803000  MOV EAX,DWORD PTR DS:[EAX+3D8]
00485E96  .   33D2          XOR EDX,EDX
00485E98  .   E8 8755FAFF   CALL icontoy.0042B424
00485E9D  .   A1 80D24800   MOV EAX,DWORD PTR DS:[48D280]
00485EA2  .   8B00          MOV EAX,DWORD PTR DS:[EAX]
00485EA4  .   33D2          XOR EDX,EDX
00485EA6  .   8950 0C       MOV DWORD PTR DS:[EAX+C],EDX
00485EA9  .v  EB 0C         JMP SHORT icontoy.00485EB7
00485EAB  >   C605 00E9480  MOV BYTE PTR DS:[48E900],0
00485EB2  .   E8 F1110000   CALL icontoy.004870A8
00485EB7  >   33C0          XOR EAX,EAX
```

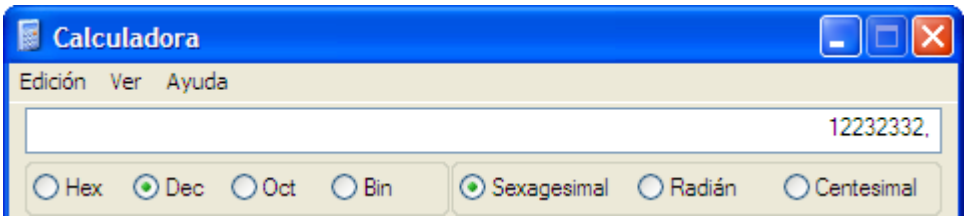
¿pero qué es esto? Madre mía está comparando el contenido de EAX (nuestro serial) así sin más con muchos valores distintos cada vez, si se cumple que son iguales salta hacia :48E52F que es donde comienza la zona del "Registered for: " jejeje… está claro que son seriales válidos TODOS..!!

Vista esta locura de programación que nunca se debe de hacer, podemos apuntarnos cualquier numerito de estos pasarlo a decimal y tenemos el serial válido..!!

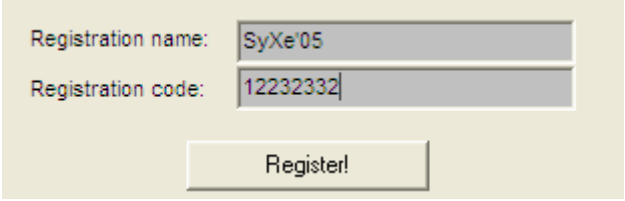
Venga pillemos uno al azar, por ejemplo el primero "0BAA68C" lo pasamos a decimal: Abrimos la calculadora de Window$ si no tenemos la de cybult que va muy bien pero no la encuentro ¿? Y escribimos el sreial en hexa:



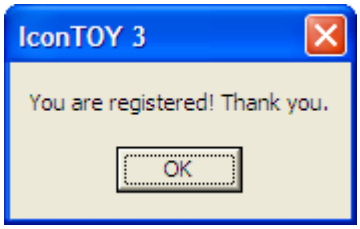
Este es el serial tal cual lo vemos en Olly pero no está en el formato adecuado pues debe estar en decimal y no en hexa, no importa lo pasamos fácil:



Ahora sí está en decimal y así mismo lo podemos probar:



Una vez puesto como en la imagen pulsamos el botón "Register!" y nos llevamos una agradable sorpresa, jejeje…



Estaba claro que eran seriales válidos TODOS los que hemos visto en la lista anterior, repito que es un suicidio hacerlo así, nunca se deben dejar estos datos al descubierto, supongo yo que este será un programa ya viejo sino no lo entiendo ¿?

Y hasta aquí el tema de hoy, nunca habíamos tocado los hardcoded serial y era un buen momento, encontré este programa y me vino de perlas ;-)

Ya saben, para cualquier duda y/o sugerencia o lo que sea escribir a:

syxe00@yahoo.es

Espero les haya gustado el tute, o al menos os haya entretenido un rato, hasta la próxima y que sea bien pronto, saludos a todo CracksLatinos SIN EXCEPCIÓN xDDDD